Experiment 7: Travelling Salesman Problem

Aim

To implement the Travelling Salesman Problem (TSP) using Python and determine the shortest possible route that visits every city exactly once and returns to the starting city.

Algorithm — Brute Force Approach

Start with a set of cities and their distances.

Select one city as the starting city.

Generate all possible permutations of the remaining cities.

For each permutation:

Calculate the total distance of the route.

Add the distance from the last city back to the starting city.

Compare the distances of all possible routes.

Select the route having the minimum total distance.

Display the optimal route and minimum distance.

Code

```
    from itertools import permutations

    # Cities: A, B, C, D

    distance = [

    [0, 10, 15, 20],

    [10, 0, 35, 25],

    [15, 35, 0, 30],

    [20, 25, 30, 0]

]

    cities = ['A', 'B', 'C', 'D']

    start = 0
```

```
    min_distance = float('inf')
    best_route = None
    # Generate all possible routes
for perm in permutations(range(1, len(cities))):
route = (start,) + perm + (start,)
total_distance = 0
for i in range(len(route) - 1):
total_distance += distance[route[i]][route[i + 1]]
   # Find minimum distance
   if total_distance < min_distance:
      min_distance = total_distance
      best_route = route
# Display result
print("Travelling Salesman Problem")
print("--------------------------------")
print("Optimal Route:", end=" ")
for i in best_route:
   print(cities[i], end=" ")
print("\nMinimum Distance:", min_distance)
```

Output

```
C:\Users\DELL>python -u "c:\Users\DELL\OneDrive\Desktop\AI Lab\salesman.py"
Travelling Salesman Problem
--------------------------------
Optimal Route: A B D C A
Minimum Distance: 80
```

Result

Thus, the Travelling Salesman Problem was successfully implemented using Python. The optimal route obtained is:

A → B → D → C → A

with a minimum travelling distance of 80 units.

Experiment 8: Write the python program to implement A* algorithm

Aim

To write and implement a Python program for the A* (A-star) search algorithm to find the shortest path between a starting node and a goal node using the cost function and heuristic function.

Algorithm

Start with the initial node.

Add the initial node to the OPEN list.

Calculate f(n) = g(n) + h(n).

Select the node having the lowest f(n).

If the selected node is the goal, stop and display the path.

Otherwise, expand the selected node and find its neighboring nodes.

Calculate the cost for each neighbor.

Update the cost and parent of a neighbor if a better path is found.

Repeat the process until the goal is reached.

Display the shortest path and its total cost.

Code

```
import heapq

def a_star(graph, heuristic, start, goal):
    open_list = []
    heapq.heappush(open_list, (0, start))
    g_cost = {start: 0}
```

```
    parent = {start: None}
    while open_list:
        current_f, current = heapq.heappop(open_list)
        if current == goal:
            path = []
            while current is not None:
                path.append(current)
                current = parent[current]
            path.reverse()
            return path, g_cost[goal]
        for neighbor, cost in graph[current]:
            new_g_cost = g_cost[current] + cost
            if neighbor not in g_cost or new_g_cost < g_cost[neighbor]:
                g_cost[neighbor] = new_g_cost
                parent[neighbor] = current
                f_cost = new_g_cost + heuristic[neighbor]
                heapq.heappush(open_list, (f_cost, neighbor))
    return None, float("inf")
graph = {
    'A': [('B', 1), ('C', 4)],
    'B': [('A', 1), ('C', 2), ('D', 5)],
    'C': [('A', 4), ('B', 2), ('D', 1)],
    'D': [('B', 5), ('C', 1), ('E', 3)],
    'E': [('D', 3)]
}
```

```
heuristic = {
    'A': 7,
    'B': 6,
    'C': 2,
    'D': 1,
    'E': 0
}
start = 'A'
goal = 'E'
path, cost = a_star(graph, heuristic, start, goal)
print("A* Algorithm")
print("----------------------")
print("Start Node:", start)
print("Goal Node:", goal)
print("Shortest Path:", " -> ".join(path))
print("Total Cost:", cost)
```

Output

```
C:\Users\DELL>python -u "c:\Users\DELL\OneDrive\Desktop\AI Lab
A* Algorithm
----------------------
Start Node: A
Goal Node: E
Shortest Path: A -> B -> C -> D -> E
Total Cost: 7

C:\Users\DELL>
```

Result

Thus, the A* algorithm was successfully implemented using Python. The shortest path from the starting node A to the goal node E is:

A → B → C → D → E

The minimum path cost is 7.

Experiment 9: Write the python program for Map Coloring to implement CSP

Aim

To write and implement a Python program for Map Coloring using Constraint Satisfaction Problem (CSP), where adjacent regions are assigned different colors.

Algorithm

Define the regions of the map.

Define the neighboring regions using constraints.

Define a set of available colors.

Select an uncolored region.

Assign a color to the selected region.

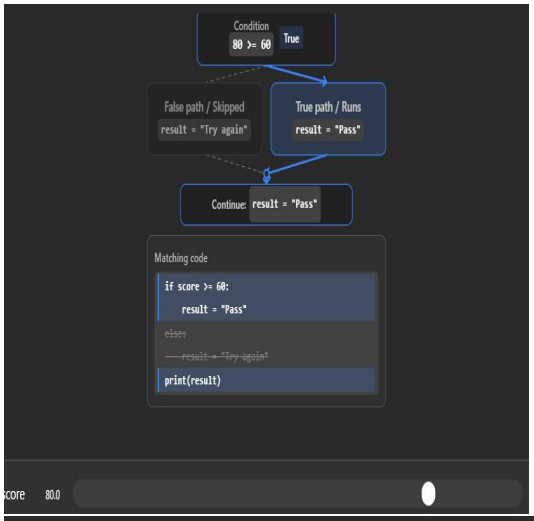
Check whether the assigned color conflicts with any neighboring region.

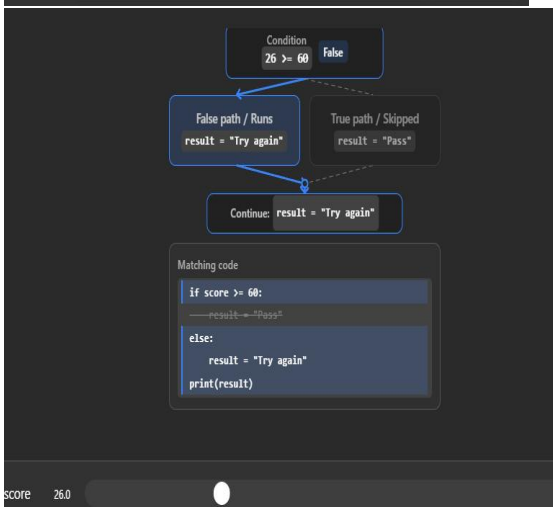
If there is no conflict, continue with the next region.

If a conflict occurs, backtrack and try another color.

Continue until all regions are colored.

Display the final map coloring solution.





Code

```
# Map regions

regions = ['A', 'B', 'C', 'D']

colors = ['Red', 'Green', 'Blue']

neighbors = {
```

```
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}
def is_valid(region, color, assignment):
    for neighbor in neighbors[region]:
        if neighbor in assignment and assignment[neighbor] == color:
            return False
    return True
def map_coloring(assignment):
    if len(assignment) == len(regions):
        return assignment
    region = next(r for r in regions if r not in assignment)
    for color in colors:
        if is_valid(region, color, assignment):
            assignment[region] = color
            result = map_coloring(assignment)
            if result:
                return result
            del assignment[region]
    return None
solution = map_coloring({})
print("Map Coloring using CSP")
print("----------------------")
```

```
if solution:
    for region, color in solution.items():
        print(region, ":", color)
else:
    print("No solution exists.")
```

Output

```
C:\Users\DELL>python -u "c:\Users\DELL\OneDrive\Desktop\AI Lab\
Map Coloring using CSP
----------------------
A : Red
B : Green
C : Blue
D : Red
```

Result

Thus, the Map Coloring problem was successfully implemented using the Constraint Satisfaction Problem (CSP) technique in Python.

The obtained coloring is:

A → Red

B → Green

C → Blue

D → Red

No two adjacent regions have the same color, so the CSP constraints are satisfied successfully.

Experiment 10: Write the python program for Tic Tac Toe game

Aim

To write and implement a Python program for the Tic Tac Toe game, where two players take turns placing X and O on a 3×3 board, and the player who gets three symbols in a row, column, or diagonal wins.

Algorithm

Create a 3×3 Tic Tac Toe board.

Initialize all positions as empty.

Display the board.

Set the first player as X.

Ask the player to enter a position from 1 to 9.

Check whether the selected position is empty.

Place the player's symbol in that position.

Check whether the player has three symbols in a row, column, or diagonal.

If a player wins, display the winner and stop the game.

If all positions are filled without a winner, declare the game as a draw.

Otherwise, switch to the other player and continue.

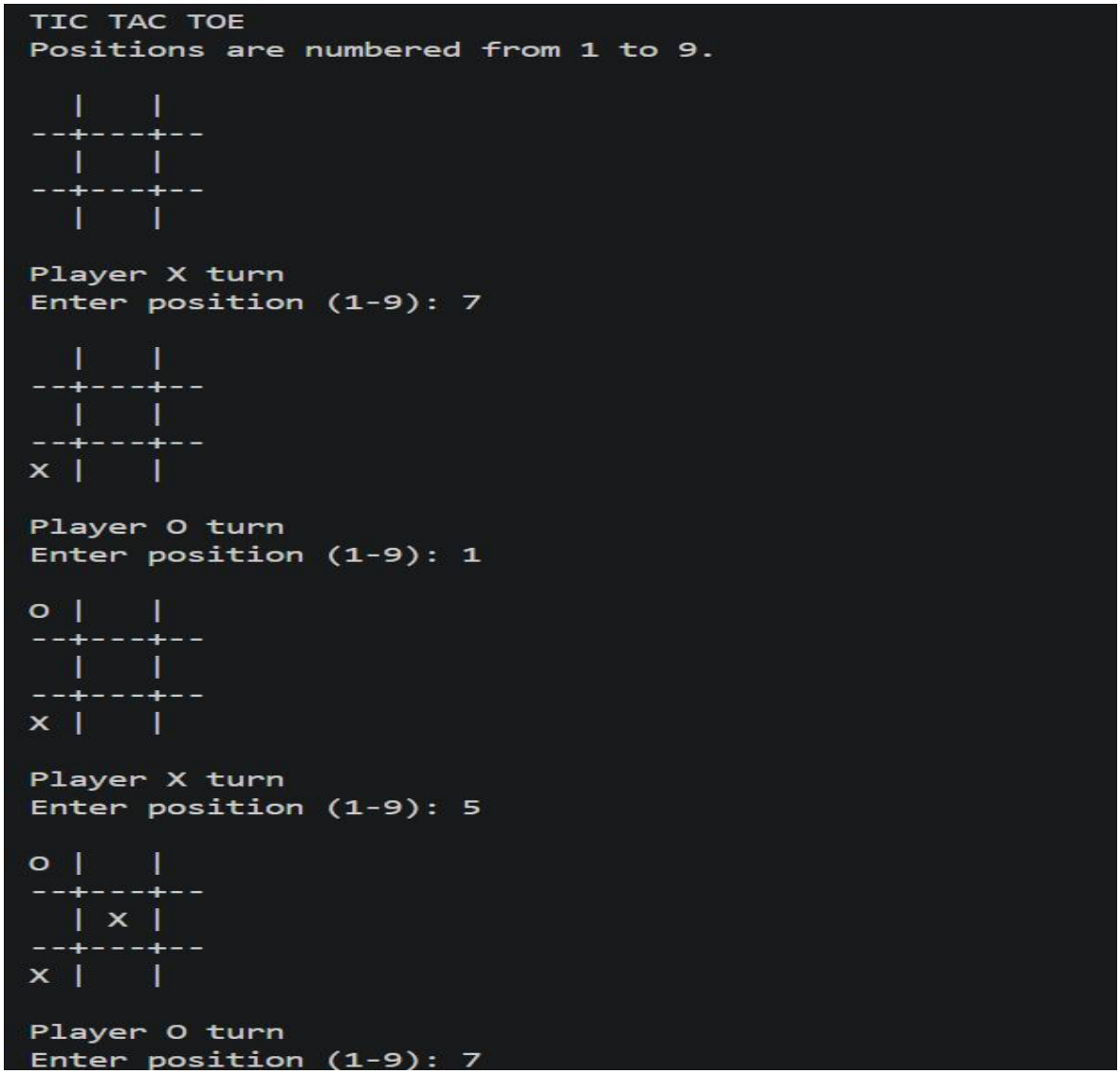
Code

```
# Tic Tac Toe Game

board = [' ' for _ in range(9)]

def display_board():

print()

print(board[0], '|', board[1], '|', board[2])

 print('--+---+--')

 print(board[3], '|', board[4], '|', board[5])

 print('--+---+--')

 print(board[6], '|', board[7], '|', board[8])
```

```
 print()
def check_winner(player):
winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
player = 'X'
print("TIC TAC TOE")
print("Positions are numbered from 1 to 9.")
for turn in range(9):
    display_board()
    print("Player", player, "turn")
while True:
        try:
            position = int(input("Enter position (1-9): "))
```

```
            if position < 1 or position > 9:
                print("Please enter a number between 1 and 9.")
            elif board[position - 1] != ' ':
                print("Position already occupied. Try again.")
            else:
                break
        except ValueError:
            print("Please enter a valid number.")
    board[position - 1] = player
    if check_winner(player):
        display_board()
        print("Player", player, "wins!")
        break
    player = 'O' if player == 'X' else 'X'
else:
    display_board()
    print("The game is a draw!")7
```

Output



Result

Thus, the Tic Tac Toe game was successfully implemented using Python. The program allows two players to play alternately, checks all possible winning combinations, and displays the winner or draw result.

Experiment 11: Write the python program to implement Minimax algorithm for gaming

Aim

To write and implement a Python program for the Minimax algorithm in gaming. The algorithm helps the computer choose the best possible move by evaluating all possible game states.

Algorithm

Create a Tic-Tac-Toe game board.

Define the winning combinations.

Check whether the current game state is a win, loss, or draw.

Generate all possible moves.

For each possible move, recursively call the Minimax function.

Assign scores:

+10 if the computer wins.

-10 if the human player wins.

0 if the game is a draw.

The computer selects the move with the maximum score.

The human player is considered the minimizing player.

Continue until the game reaches a final state.

Display the winner or draw.

Code

```
# Minimax Algorithm for Gaming

board = [' ' for _ in range(9)]

def display_board():
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
    print('--+---+--')
    print(board[6], '|', board[7], '|', board[8])
    print()

def check_winner(player):
    winning_positions = [
```

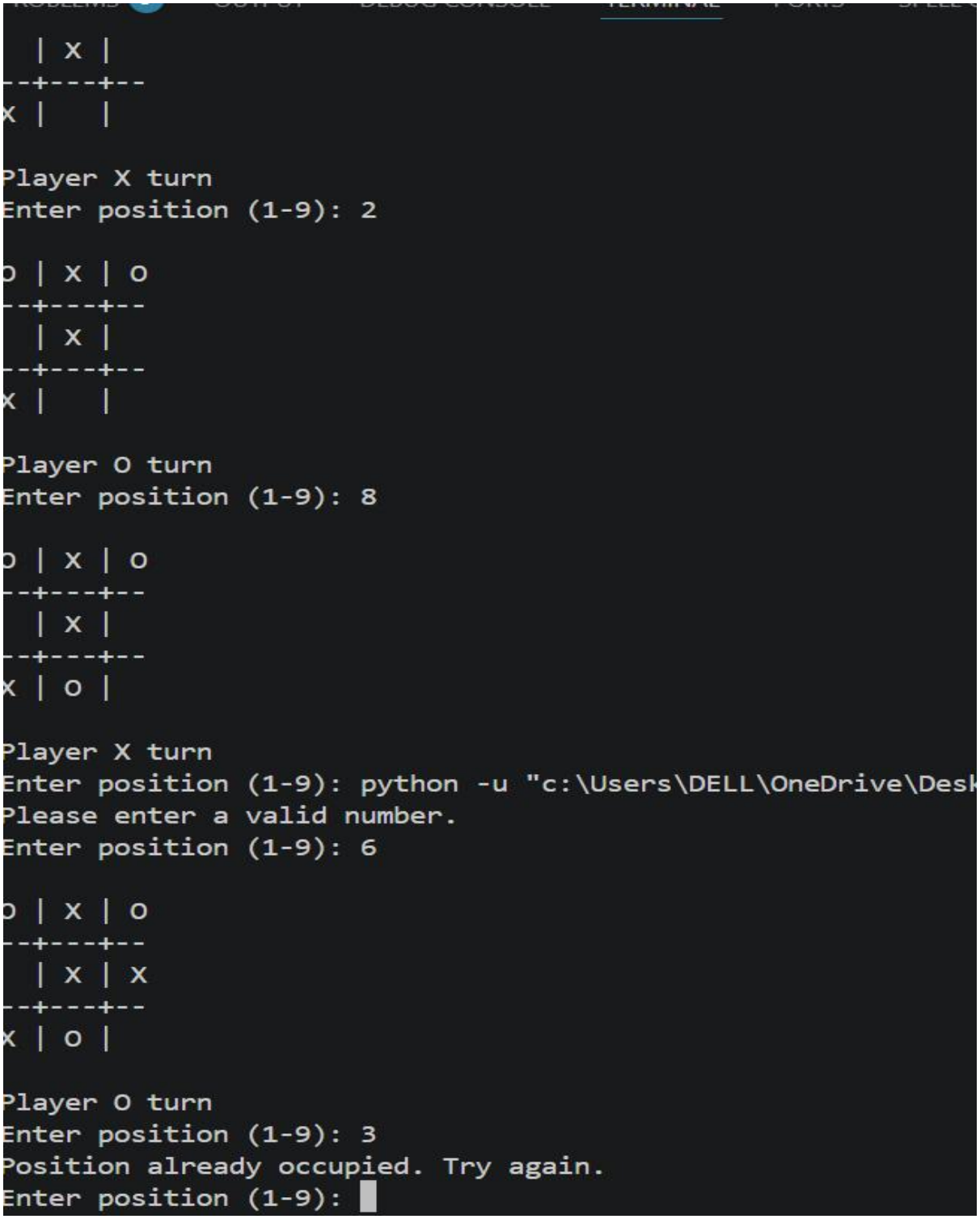
```
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
        def is_full():
    return ' ' not in board
def minimax(is_maximizing):
    if check_winner('O'):
        return 10
    if check_winner('X'):
        return -10
    if is_full():
        return 0
    if is_maximizing:
        best_score = -float('inf')
        for i in range(9):
```

```
            if board[i] == ' ':
                board[i] = 'O'
                score = minimax(False)
                board[i] = ' '
                best_score = max(best_score, score)
        return best_score
    # Minimizing player - Human
    else:
        best_score = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
                score = minimax(True)
                board[i] = ' '
                best_score = min(best_score, score)
        return best_score
def find_best_move():
    best_score = -float('inf')
    best_move = -1
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = minimax(False)
            board[i] = ' '
            if score > best_score:
```

```
            best_score = score
            best_move = i
    return best_move
print("TIC TAC TOE - MINIMAX")
print("You = X")
print("Computer = O")
while True:
    display_board()
    while True:
        try:
            position = int(input("Enter position (1-9): "))
            if position < 1 or position > 9:
                print("Enter a number from 1 to 9.")
            elif board[position - 1] != ' ':
                print("Position already occupied.")
            else:
                break
        except ValueError:
            print("Enter a valid number.")
    board[position - 1] = 'X'
    # Check human winner
    if check_winner('X'):
        display_board()
        print("Human wins!")
        break
```

```
    if is_full():
        display_board()
        print("Game Draw!")
        break
    move = find_best_move()
    board[move] = 'O'
    print("Computer chooses position:", move + 1)
    if check_winner('O'):
        display_board()
        print("Computer wins!")
        break
    if is_full():
        display_board()
        print("Game Draw!")
        break
```

Output



Result

Thus, the Minimax algorithm was successfully implemented using Python for gaming. The program evaluates the possible game states and enables the computer to select the optimal move. The program successfully identifies whether the human wins, computer wins, or the game ends in a draw.

Experiment 12: Write the python program to implement Alpha & Beta pruning algorithm for gaming

Aim

To write and implement a Python program for the Alpha-Beta Pruning algorithm in gaming and determine the optimal move by eliminating branches that do not affect the final decision.

Algorithm

Create a Tic-Tac-Toe game board.

Define all possible winning combinations.

Set the initial values:

Alpha = -∞

Beta = +∞

Generate all possible moves.

Apply the Minimax algorithm to evaluate each possible move.

For the maximizing player:

Select the maximum score.

Update alpha.

For the minimizing player:

Select the minimum score.

Update beta.

If beta <= alpha, stop exploring that branch. This is called pruning.

Select the move with the best score.

Continue until the game ends.

Display the winner or draw.

Code

```
board = [' ' for _ in range(9)]
```

```
def display_board():
    print()
    print(board[0], '|', board[1], '|', board[2])
    print('--+---+--')
    print(board[3], '|', board[4], '|', board[5])
    print('--+---+--')
    print(board[6], '|', board[7], '|', board[8])
    print()
def check_winner(player):
    winning_positions = [
        (0, 1, 2),
        (3, 4, 5),
        (6, 7, 8),
        (0, 3, 6),
        (1, 4, 7),
        (2, 5, 8),
        (0, 4, 8),
        (2, 4, 6)
    ]
    for a, b, c in winning_positions:
        if board[a] == board[b] == board[c] == player:
            return True
    return False
def is_full():
    return ' ' not in board
```

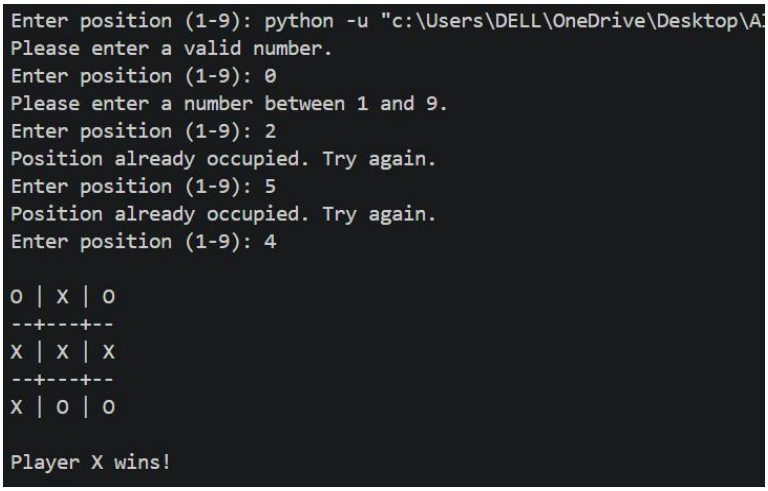
```
def alpha_beta(is_maximizing, alpha, beta):
    if check_winner('O'):
        return 10
    # Human wins
    if check_winner('X'):
        return -10
    if is_full():
        return 0
    if is_maximizing:
        best_score = -float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'O'
                score = alpha_beta(False, alpha, beta)
                board[i] = ' '
                best_score = max(best_score, score)
                alpha = max(alpha, best_score)
                if beta <= alpha:
                    break
        return best_score
    else:
        best_score = float('inf')
        for i in range(9):
            if board[i] == ' ':
                board[i] = 'X'
```

```
            score = alpha_beta(True, alpha, beta)
            board[i] = ' '
            best_score = min(best_score, score)
            beta = min(beta, best_score)
            if beta <= alpha:
                break
        return best_score
def find_best_move():
    best_score = -float('inf')
    best_move = -1
    alpha = -float('inf')
    beta = float('inf')
    for i in range(9):
        if board[i] == ' ':
            board[i] = 'O'
            score = alpha_beta(False, alpha, beta)
            board[i] = ' '
            if score > best_score:
                best_score = score
                best_move = i
    return best_move
print("TIC TAC TOE - ALPHA-BETA PRUNING")
print("You = X")
print("Computer = O")
while True:
```

```
display_board()
while True:
    try:
        position = int(input("Enter position (1-9): "))
        if position < 1 or position > 9:
            print("Enter a number between 1 and 9.")
        elif board[position - 1] != ' ':
            print("Position already occupied.")
        else:
            break
    except ValueError:
        print("Enter a valid number.")
board[position - 1] = 'X'
if check_winner('X'):
    display_board()
    print("Human wins!")
    break
if is_full():
    display_board()
    print("Game Draw!")
    break
move = find_best_move()
board[move] = 'O'
print("Computer chooses position:", move + 1)
if check_winner('O'):
```

```
        display_board()
        print("Computer wins!")
        break
    if is_full():
        display_board()
        print("Game Draw!")
        break
```

Output



Result

Thus, the Alpha-Beta Pruning algorithm was successfully implemented using Python for gaming. The program evaluates the possible moves and eliminates unnecessary branches using Alpha and Beta values, allowing the computer to efficiently select the optimal move.

Experiment 13: Write the python program to implement decision tree

Aim

To write and implement a Python program for Decision Tree classification and predict the class of new data using the Decision Tree algorithm.

Algorithm

Import the required Python libraries.

Create a dataset containing input features and target classes.

Separate the dataset into input (X) and output (y).

Create a Decision Tree Classifier.

Train the classifier using the training data.

Provide a new sample for prediction.

Use the trained Decision Tree to predict its class.

Display the predicted result.

Code

```
from sklearn.tree import DecisionTreeClassifier
X = [
    [1, 1],
    [1, 0],
    [0, 1],
    [0, 0],
    [1, 1],
    [0, 0]
]
y = ['Yes', 'Yes', 'No', 'No', 'Yes', 'No']
model = DecisionTreeClassifier(criterion='entropy', random_state=0)
model.fit(X, y)
test = [[1, 1]]
prediction = model.predict(test)
print("Decision Tree")
print("----------------")
print("Test Data:", test)
print("Predicted Class:", prediction[0])
```

Output

```
C:\Users\DELL>python -u "c:\Users\DELL\OneDrive\Desktop\decision_tree.py"
Decision Tree
---------------
Test Data: [[1, 1]]
Predicted Class: Yes
```

Result

Thus, the Decision Tree classification algorithm was successfully implemented using Python. The trained model successfully classified the given test data and predicted the class as Yes.

Experiment 14: Write the python program to implement Feed forward neural Network

Aim

To write and implement a Python program for a Feed Forward Neural Network (FFNN) to classify input data into different classes.

Algorithm

Import the required libraries.

Create the input dataset and target values.

Divide the dataset into training and testing data.

Create a Feed Forward Neural Network using MLPClassifier.

Train the network using the training data.

Give new input data to the trained network.

Predict the class of the input.

Display the predicted result.

Code

```
from sklearn.neural_network import MLPClassifier

X = [
```

```
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]
y = [0, 0, 0, 1]
model = MLPClassifier(
    hidden_layer_sizes=(5,),
    activation='relu',
    solver='lbfgs',
    max_iter=5000,
    random_state=42
)
model.fit(X, y)
test_data = [
    [0, 0],
    [0, 1],
    [1, 0],
    [1, 1]
]
prediction = model.predict(test_data)
print("Feed Forward Neural Network")
print("----------------------------")
for i in range(len(test_data)):
    print("Input:", test_data[i],
```

"Predicted Output:", prediction[i])

Output

```
C:\Users\DELL>python -u "c:\Users\DELL\OneDrive\Desktop
Feed Forward Neural Network
--------------------------
Input: [0, 0] Predicted Output: 0
Input: [0, 1] Predicted Output: 0
Input: [1, 0] Predicted Output: 0
Input: [1, 1] Predicted Output: 1
```

Result

Thus, the Feed Forward Neural Network was successfully implemented using Python. The trained neural network successfully classified the test input [1, 1] and predicted the output as 1.

Experiment 16: Sum of Integers from 1 to N

Aim

To write and execute a Prolog program to find the sum of integers from 1 to N using recursion.

Algorithm

Start with N and initialize the sum.

Define the base case: when N = 0, the sum is 0.

For N > 0, calculate the sum of integers from 1 to N-1.

Add N to the obtained sum.

Continue recursively until N = 0.

Display the final sum.

Program

% Sum of integers from 1 to N

sum_n(0, 0).

```
sum_n(N, Sum) :-
   N > 0,
   N1 is N - 1,
   sum_n(N1, S1),
   Sum is N + S1.
```

Output

```
23 ?- consult('C:/Users/RAMANI/Documents/Prolog/sum.pl').
true.

23 ?- sum_n(10, S).
S = 55
```

Result

Thus, the Prolog program to find the sum of integers from 1 to N was successfully implemented and executed.

Experiment 17: Database with Name and DOB

Aim

To implement a Prolog database containing the name and date of birth (DOB) of persons and retrieve the information using queries.

Algorithm

Define facts containing the name and date of birth of persons.

Define a predicate to retrieve the DOB using the person's name.

Load the Prolog program.

Enter a query with the person's name.

Display the corresponding date of birth.

Program

```
% Database of Name and Date of Birth

person(ramani, date(15, 5, 2006)).
```

person(rahul, date(10, 8, 2005)).

person(sita, date(25, 12, 2006)).

person(raj, date(5, 3, 2005)).

person(anita, date(18, 7, 2006)).

Output

```
24 ?- consult('C:/Users/RAMANI/Documents/Prolog/dob.pl').
true.

25 ?- dob(ramani, DOB).
DOB = date(15, 5, 2006).

26 ?- dob(Name, date(10, 8, 2005)).
Name = rahul.

27 ?- person(Name, DOB).
Name = ramani,
DOB = date(15, 5, 2006) ;
Name = rahul,
DOB = date(10, 8, 2005) ;
Name = sita,
DOB = date(25, 12, 2006) ;
Name = raj,
DOB = date(5, 3, 2005) ;
Name = anita,
DOB = date(18, 7, 2006).
```

Result

Thus, the Prolog database for storing and retrieving Name and DOB was successfully implemented.

Experiment 18: Student, Teacher, Subject and Code Database

Aim

To implement a Prolog database containing information about students, teachers, subjects and subject codes and retrieve the information using queries.

Algorithm

Define facts for students and their departments.

Define facts for teachers and subjects.

Define facts for subjects and their corresponding codes.

Define the subjects studied by students.

Load the database into Prolog.

Execute queries to retrieve the required information.

Display the results.

Program

```
% Student Database

student(ramani, ece).

student(rahul, cse).

student(sita, eee).

student(raj, ece).

% Teacher Database

teacher(manikandan, ai).

teacher(priya, dbms).

teacher(kumar, networks).

teacher(anitha, python).

% Subject and Subject Code

subject(ai, cs401).

subject(dbms, cs402).

subject(networks, cs403).

subject(python, cs404).
```

% Student studies subject

studies(ramani, ai).

studies(ramani, python).

studies(rahul, dbms).

studies(sita, networks).

studies(raj, ai).

Output

```
29 ?- consult('C:/Users/RAMANI/Documents/Prolog/student.pl').
true.

30 ?- student(ramani, Department).
Department = ece.

31 ?- student(ramani, Department).
Department = ece.

31 ?- teacher(manikandan, Subject).
Subject = ai.

32 ?- studies(ramani, Subject).
Subject = ai ;
Subject = python.
```

Result

Thus, the Prolog database for Student, Teacher, Subject and Code was successfully implemented and queried.

Experiment 19: Planets Database

Aim

To implement a Prolog database containing information about planets, their positions and their types.

Algorithm

Define facts representing the planets.

Define the position of each planet from the Sun.

Define the type of each planet.

Load the database into Prolog.

Use queries to retrieve planet information.

Display the required results.

Program

```
% Planet Database

planet(mercury).

planet(venus).

planet(earth).

planet(mars).

planet(jupiter).

planet(saturn).

planet(uranus).

planet(neptune).

% Planet and its position from the Sun

position(mercury, 1).

position(venus, 2).

position(earth, 3).

position(mars, 4).

position(jupiter, 5).

position(saturn, 6).

position(uranus, 7).

position(neptune, 8).


% Planet and its type

type(mercury, terrestrial).
```

type(venus, terrestrial).

type(earth, terrestrial).

type(mars, terrestrial).

type(jupiter, gas_giant).

type(saturn, gas_giant).

type(uranus, ice_giant).

type(neptune, ice_giant).

Output

```
34 ?- consult('C:/Users/RAMANI/Documents/Prolog/planets.pl').
true.

35 ?- planet(earth).
true.

36 ?- position(earth, P).
P = 3.

37 ?- type(jupiter, T).
T = gas_giant.

38 ?- position(X, P).
X = mercury,
P = 1 ;
X = venus,
P = 2 ;
X = earth,
P = 3 ;
X = mars,
P = 4 ;
X = jupiter,
P = 5 ;
X = saturn,
P = 6 ;
X = uranus,
P = 7 ;
X = neptune,
P = 8.
```

Result

Thus, the Prolog program for storing and retrieving information about planets was successfully implemented and executed.

Experiment 20: Towers of Hanoi

Aim

To implement the Towers of Hanoi problem using Prolog recursion.

Algorithm

Start with N disks on the source peg.

If there is only one disk, move it directly from the source to the destination.

For more than one disk, move N-1 disks from the source to the auxiliary peg.

Move the largest disk from the source to the destination peg.

Move the N-1 disks from the auxiliary peg to the destination peg.

Repeat recursively until all disks are moved.

Display each disk movement.

Program

```
% Towers of Hanoi

hanoi(1, Source, Destination, _) :-

    write('Move disk from '),

    write(Source),

    write(' to '),

    write(Destination),

    nl.

hanoi(N, Source, Destination, Auxiliary) :-

    N > 1,

    N1 is N - 1,

    hanoi(N1, Source, Auxiliary, Destination),
```

hanoi(1, Source, Destination, Auxiliary),

hanoi(N1, Auxiliary, Destination, Source).

Output

```
39 ?- consult('C:/Users/RAMANI/Documents/Prolog/hanoi.pl').
true.

40 ?- hanoi(3, a, c, b).
Move disk from a to c
Move disk from a to b
Move disk from c to b
Move disk from a to c
Move disk from b to a
Move disk from b to c
Move disk from a to c
true ;
```

Result

Thus, the Towers of Hanoi problem was successfully implemented using Prolog recursion.